

CSCI-GA 3033: Honors Machine Learning

Assignment 1

Released: September 21, 2026. Due: October 12, 2026.

Submit on Gradescope.

Submit your writeup in pdf and all source code in a zip file (with proper documentation). Write a script for each programming exercise so that the TAs can easily run and verify your results. Make sure your code runs!

Submit your written solutions and code on Gradescope.

[Text in square brackets are hints that can be ignored.]

Regression datasets are in `data/regression/`; classification datasets are in `data/classification/`. Use NumPy and plotting tools for your implementations; use model-fitting libraries only where explicitly allowed.

Exercise 1: Regression Implementation (8 pts)

Recall that ridge regression refers to

$$\min_{\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}} \underbrace{\frac{1}{2n} \|X\mathbf{w} + b\mathbf{1} - \mathbf{y}\|_2^2}_{\text{error}} + \underbrace{\lambda \|\mathbf{w}\|_2^2}_{\text{loss}}, \quad (1)$$

where $X \in \mathbb{R}^{n \times d}$ and $\mathbf{y} \in \mathbb{R}^n$ are the given dataset and $\lambda \geq 0$ is the regularization hyperparameter. If $\lambda = 0$, then this is the standard linear regression problem. Observe the distinction between the error (which does not include the regularization term) and the loss (which does).

- (0.5 pts) Show that ridge regression can be rewritten as a non-regularized linear regression problem. That is, prove 1 is equivalent to

$$\min_{\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}} \frac{1}{2n} \left\| \begin{bmatrix} X & \mathbf{1}_n \\ \sqrt{2\lambda n} I_d & \mathbf{0}_d \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} - \begin{bmatrix} \mathbf{y} \\ \mathbf{0}_d \end{bmatrix} \right\|_2^2, \quad (2)$$

where I_d is the d -dimensional identity matrix, and $\mathbf{0}_k$ and $\mathbf{1}_k$ are zero and one column vectors in k dimensions.

Ans:

- (0.5 pts) Show that the derivatives of 1 are

$$\frac{\partial}{\partial \mathbf{w}} = \frac{1}{n} X^\top (X\mathbf{w} + b\mathbf{1} - \mathbf{y}) + 2\lambda \mathbf{w} \quad (3)$$

$$\frac{\partial}{\partial b} = \frac{1}{n} \mathbf{1}^\top (X\mathbf{w} + b\mathbf{1} - \mathbf{y}). \quad (4)$$

Ans:

- (2 pts) Implement ridge regression using the closed form solution for linear regression as derived in lecture. You may find the function `numpy.linalg.solve` useful.

Ans:

- (2 pts) Implement the gradient descent algorithm for solving ridge regression. The following incomplete pseudo-code may be of help. Your training loss should monotonically decrease during iteration; if not try to tune your step size η , e.g. make it smaller.

Ans:

Algorithm 1: Gradient descent for ridge regression.

Input: $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$, $w_0 = \mathbf{0}_d$, $b_0 = 0$, $\text{max_pass} \in \mathbb{N}$, $\eta > 0$, $\text{tol} > 0$
Output: w, b

```

1 for  $t = 1, 2, \dots, \text{max\_pass}$  do
2    $w_t \leftarrow$ 
3    $b_t \leftarrow$ 
4   if  $\|w_t - w_{t-1}\| \leq \text{tol}$  then           // can use other stopping criteria
5     break
6  $w \leftarrow w_t$ ,  $b \leftarrow b_t$ 

```

Ans:

5. (3 pts) Test your two implementations on the Boston **housing** dataset (to predict the median house price, i.e., y). Use the train and test splits provided in **data/regression/**. Try $\lambda \in \{0, 10\}$ and report your training error, training loss and test error for each. You may have trouble getting gradient descent to work if you don't **standardize** your data (you may not use sklearn to standardize your data, do it yourself). Compute the feature means and standard deviations using only the training set, and use these same quantities to standardize both training and test features. When comparing your closed-form and gradient-descent implementations, use the same preprocessing for both. For $\lambda > 0$, standardizing the features can change the optimal predictions even when λ is held fixed. (Try gradient descent both without and with standardizing the data and see how it differs.) For unstandardized data, you may reach your iteration limit before convergence. Report this if it occurs. For the gradient descent algorithm, plot the training loss over iterations. Compare the running time of the two approaches, which is faster? Overall, which approach do you think is better? Explain why.

Ans:

Exercise 2: Playing with Regression (5 pts)

You may use the Python package **scikit-learn** for this exercise; other exercises state their own library permissions.

Train (unregularized) linear regression, ridge regression, and lasso on the mystery datasets A, B, and C in **data/regression/** (using `X_train` and `Y_train` for each dataset). When the unregularized least-squares solution is nonunique, use the solution with minimum Euclidean norm of the feature coefficient vector (excluding the intercept). This is the default behavior of scikit-learn's **LinearRegression**. For ridge regression and lasso, use k-fold cross validation to determine appropriate choices of the hyperparameters (you may NOT use functions like `RidgeCV` and `LassoCV`, you must implement it yourself). Report your number of folds, candidate regularization parameters, and any preprocessing. Select hyperparameters using only the training data. If you standardize, compute the standardization statistics separately within each training fold. Report the average mean squared error on the test set for each method, as well as the selected regularization parameters. Which approach performs best in each case? For each dataset (A, B, C), do the following: Create a histogram where the x-axis is divided into bins corresponding to the values of coordinates of the parameter vector, and the y-axis is the number of coordinates of the parameter vector which fall into each bin. Plot the three parameter vectors (one for unregularized, one for ridge, one for lasso) on the same histogram, so they're all visible at once (change the opacity setting for the bars if necessary). [There should be three total histograms, each with three parameter vectors overlaid.]

Ans:

Exercise 3: Nearest Neighbour Regression (5 pts)

1. (1 pt) Implement k -nearest neighbour regression, for a dataset of n X, y pairs where $X \in \mathbb{R}^d$ and $y \in \mathbb{R}$. This is similar to k -nearest neighbour classification, but instead of aggregating labels by taking the majority, we average the y values of the k nearest neighbours. Use ℓ_2 distance as the distance metric, that is, the distance between points X_1 and X_2 is $\|X_1 - X_2\|_2$. State and justify the time complexity of your implementation for predicting one query.

Ans:

2. (2 pts) For the training sets of the $d = 1$ mystery datasets D and E in `data/regression/`, compute a) the (unregularized) least squares linear regression solution and b) the k -nearest neighbour regression solution with each integer k from 1 to 9. For each dataset, on one plot with the x and y axes corresponding to the x and y values, display the least squares solution, the 1-nearest neighbour solution, and the 9-nearest neighbour solution. Be sure to plot these solutions for all points between the minimum and maximum x values, not just for the points in the dataset. On another plot, with k on the x axis and test mean-squared error on the y axis, display the error of the k -NN solution for each value of k . On the same plot, include the test error of the least squares solution as a horizontal line. When does each approach perform better, and why?

Ans:

3. (2 pts) For the training set of the $d = 20$ mystery dataset F in `data/regression/`, compute a) the (unregularized) least squares linear regression solution and b) the k -nearest neighbour regression solution with each integer k from 1 to 9. Plot, with k on the x axis and test mean-squared error on the y axis, display the error of the k -NN solution for each value of k . On the same plot, include the test error of the least squares solution as a horizontal line. Which approach performs better, and why? Hint: consider inspecting distances between the test points and their k nearest neighbours in the training set.

Ans:

Exercise 4: Nearest-Neighbor Classification (4 pts)

Let $X \sim \text{Unif}[0, 1]$ and $Y \in \{0, 1\}$, and suppose

$$\Pr(Y = 1 \mid X = x) = \begin{cases} 1/4, & x \leq 1/4, \\ 3/4, & x > 1/4. \end{cases}$$

- (1 pt) Determine the Bayes classifier and its error. Ans:
- (2 pts) Now, we consider the k NN classifier on n training examples and 1 test example drawn from this distribution. Determine the limiting expected test error as $n \rightarrow \infty$ for:
 - $k = 1$;
 - $k = n$ (where k, n are odd).

Ans:

- (1 pt) As discussed in class, consistency for k -NN requires both $k(n) \rightarrow \infty$ and $k(n)/n \rightarrow 0$. Using your answers above, explain the distinct purposes of these conditions. Ans:

Exercise 5: Poisson Regression (4 pts)

Recall that in logistic regression we assumed the *binary* label $Y_i \in \{0, 1\}$ follows the Bernoulli distribution: $\Pr(Y_i = 1 \mid X_i) = p_i$, where p_i also happens to be the mean. Under the independence assumption we derived the log-likelihood function:

$$\sum_{i=1}^n (1 - y_i) \log(1 - p_i) + y_i \log(p_i). \quad (5)$$

Then, we parameterized the mean parameter p_i through the logit transform:

$$\log \frac{p_i}{1 - p_i} = \mathbf{w}^\top \mathbf{x}_i + b, \quad \text{or equivalently} \quad p_i = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x}_i - b)}. \quad (6)$$

Lastly, we found the weight vector $\mathbf{w}$ and b by maximizing the log-likelihood function.

In the following we generalize the above idea to the case where $Y_i \in \mathbb{N}$, i.e., Y_i can take any natural number (for instance, when we are interested in predicting the number of customers or network packages).

1. (1 pt) Naturally, we assume $Y_i \in \mathbb{N}$ follows the Poisson distribution (with mean $\mu_i \geq 0$):

$$\Pr(Y_i = k | X_i) = \frac{\mu_i^k}{k!} \exp(-\mu_i), \quad k = 0, 1, 2, \dots \quad (7)$$

Given a dataset $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^n$, what is the log-likelihood function (of μ_i 's) given $\mathcal{D}$?

Ans:

2. (1 pt) Can you give some justification of the parameterization below?

$$\log \mu_i = \mathbf{w}^\top \mathbf{x}_i + b. \quad (8)$$

Ans:

3. (1 pt) Based on the above, write down the objective function for Poisson regression. Please specify the optimization variables and whether you are maximizing or minimizing. [Constants can be dropped.]

Ans:

4. (1 pt) Compute the gradient of your objective function above and formulate a gradient algorithm for finding the weight vector $\mathbf{w}$ and b .

Ans:

Exercise 6: Fun with Classification (6 pts)

For this problem, you are allowed to use `statsmodels` and `sklearn` as directed.

1. (2 pts) Run logistic regression, SVM with ℓ_2 regularization with parameter 1 (soft-margin SVM), and hard-margin SVM on Mystery Dataset A in `data/classification/` (note that there is only a training dataset and no test dataset). Use `Logit` from `statsmodels` and `SVC` (with linear kernel) from `sklearn` for the soft-margin model. Include an intercept in both models. For the hard-margin model, solve the constrained quadratic program (a quadratic-programming library is allowed), or approximate it with increasing finite values of C and verify feasibility of the margin constraints. Do not pass infinity to `SVC`. Distinguish mathematical nonexistence or infeasibility from numerical warnings or library validation errors. One of these methods does not have a finite optimal solution. Identify which method and give a mathematically rigorous explanation. Your software may report an error or warning, or return a finite numerical result; explain why its output does not contradict your conclusion. [Think carefully about the loss function used for this method.] How could the associated problems be remedied? Discuss similarities and differences between the solution obtained via these two working methods.

Ans:

2. (4 pts) Take your solution for the soft-margin SVM from the previous part. For each point in the dataset, compute its signed margin $y_i(\mathbf{w}^\top \mathbf{x}_i + b)$, replacing labels 0 with -1 . How many of these values are ≤ 1 ? [Be sure you're getting all of them – there may be numerical precision issues, so if in doubt, err on the side of counting a point.] Based on your answer to these questions, sketch a 2D caricature of what the points and the hyperplane defined by the SVM solution look like. (A “caricature” in this context means a rough sketch of what you think Mystery Dataset A in `data/classification/` looks like, using the information you have learned in this question and the previous part. There will be a few key features of the dataset you can emphasize in your caricature. This might use more information than the literal exact answers to the questions that have been asked (though not much more).) Write the parameter vector solution to the SVM problem as a linear combination of the minimum possible number of points in your dataset. How many points are required? Justify why fewer points cannot suffice. [You may find built-in functions useful for this purpose.]

Compute the solution for the same three methods on Mystery Dataset B in `data/classification/`. You will again run into issues with one of them – explain why. [The answer is likely to be simpler than last time.] Find a way to write the parameter vector solution to the SVM problem as a linear combination of some points in your dataset – do not report the solution itself, but report how many points you used, and how you arrived at this answer. You may use the dual coefficients returned by your SVM solver to

obtain such a representation, counting the nonzero coefficients. You do not need to minimize the number of points in this representation. Compare the empirical prediction accuracy (i.e., using 0-1 loss) of the successfully-trained classifiers on the test set.

Ans:

- (Optional – 0 pts) Construct a dataset in which every point is a support vector. Do this for the soft margin ($C = 1$) and hard margin ($C = \infty$) cases. Make sure this dataset has $n \geq 100$ points. Submit these datasets in the `data` folder, as well as code required to generate them and demonstrate the number of support vectors.

Ans:

Exercise 7: Support Vector Regression (4 pts)

Let us consider support vector regression:

$$\min_{\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \max\{|y_i - (\mathbf{w}^\top \mathbf{x}_i + b)| - \varepsilon, 0\}, \quad (9)$$

where $\mathbf{x}_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$, and $\|\mathbf{w}\|_2 := \sqrt{\sum_{j=1}^d w_j^2}$ is the Euclidean norm. The above expression is the loss function, the error is simply the latter term $C \sum_{i=1}^n \max\{|y_i - (\mathbf{w}^\top \mathbf{x}_i + b)| - \varepsilon, 0\}$.

- (3 pts) Derive the Lagrangian dual of the support vector regression loss function (9). Please include intermediate steps so that you can get partial credit.

Ans:

- (1 pt) Compute the gradient with respect to $\mathbf{w}$ and b of the individual-example loss: Note that in places where the function is non-differentiable, you might have to compute a sub-gradient.

$$C \max\{|y_i - (\mathbf{w}^\top \mathbf{x}_i + b)| - \varepsilon, 0\} \quad (10)$$

Ans:

Exercise 8: Kernels (5 pts)

For the following questions you might find it useful to recall the definition of a matrix being positive semidefinite (PSD). A matrix $M \in \mathbb{R}^{d \times d}$ is PSD if and only if $x^T M x \geq 0$ for all vectors $x \in \mathbb{R}^d$. It may also be helpful to refresh yourself on Taylor series.

- (2 pt) For $x, y \in \mathbb{R}$ and $\alpha > 0$, consider the kernel function $k(x, y) = \exp(-\alpha(x - y)^2)$. What is the corresponding feature map $\phi(\cdot)$ such that $\phi(x)^T \phi(y) = k(x, y)$? If you were using this kernel for an SVM model, would you prefer to solve the primal or dual representation? Why?

Ans:

- (1 pt) Consider the function $\frac{1}{1-xy}$, where $x, y \in (-1, 1)$. Is this function a valid kernel? If so, write out the corresponding feature map $\phi(\cdot)$, if not, explain why.

Ans:

- (1 pt) Consider the function $\log(1 + xy)$, where $0 < x, y \in \mathbb{R}$. Is this function a valid kernel? If so, write out the corresponding feature map $\phi(\cdot)$, if not, explain why.

Ans:

- (1 pt) Consider the function $\cos(x + y)$, where $x, y \in \mathbb{R}$. Is this function a valid kernel? If so, write out the corresponding feature map $\phi(\cdot)$, if not, explain why.

Ans: